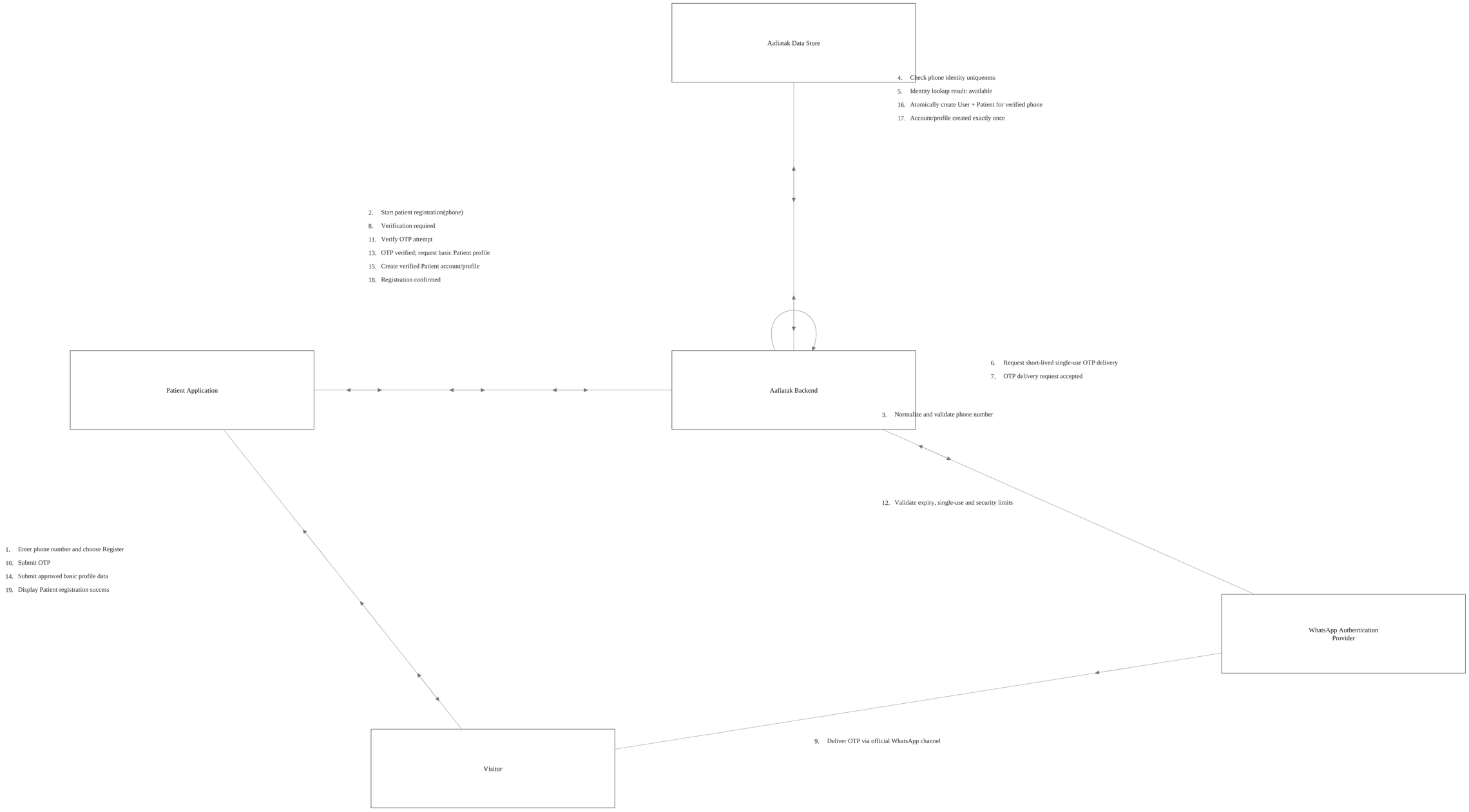
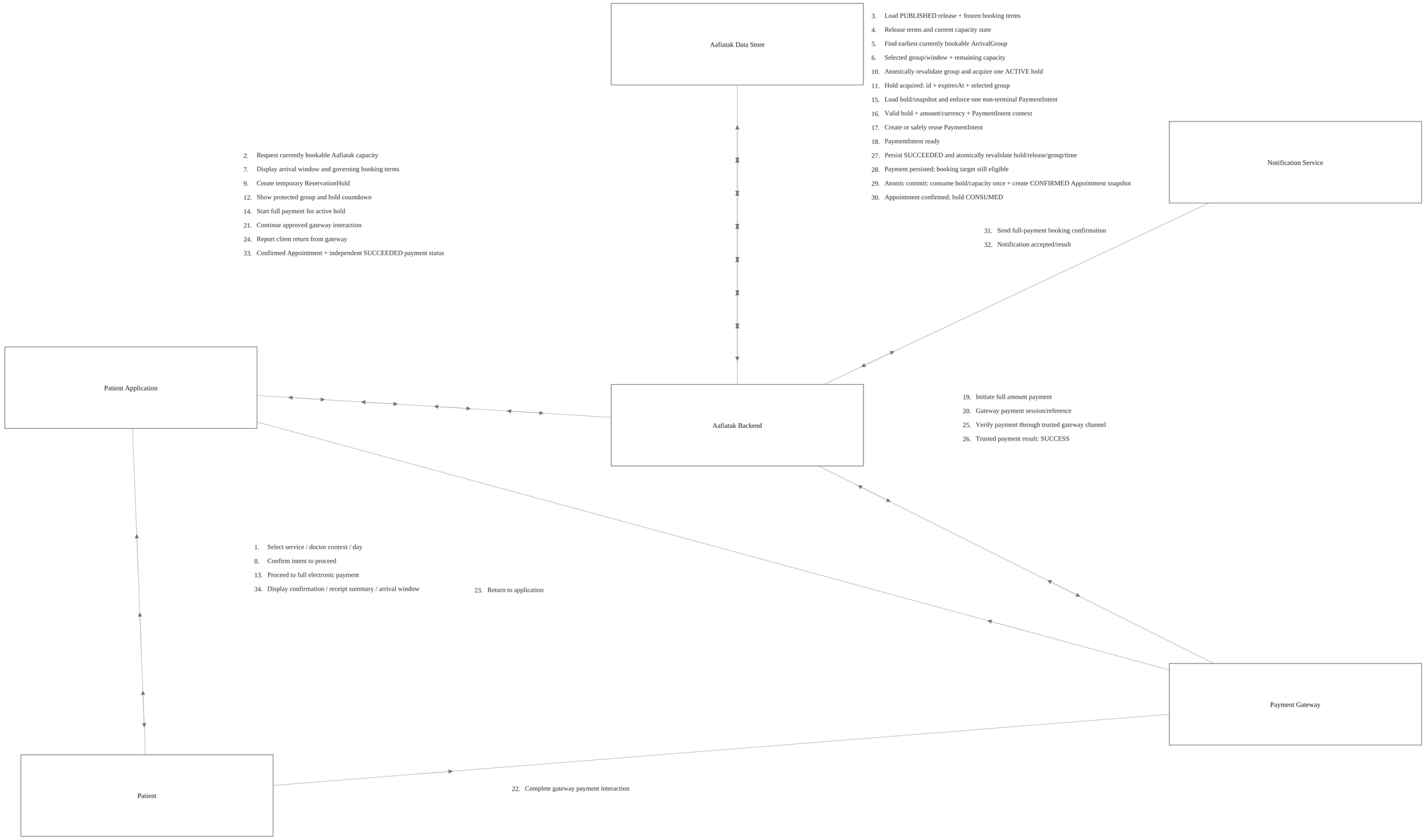
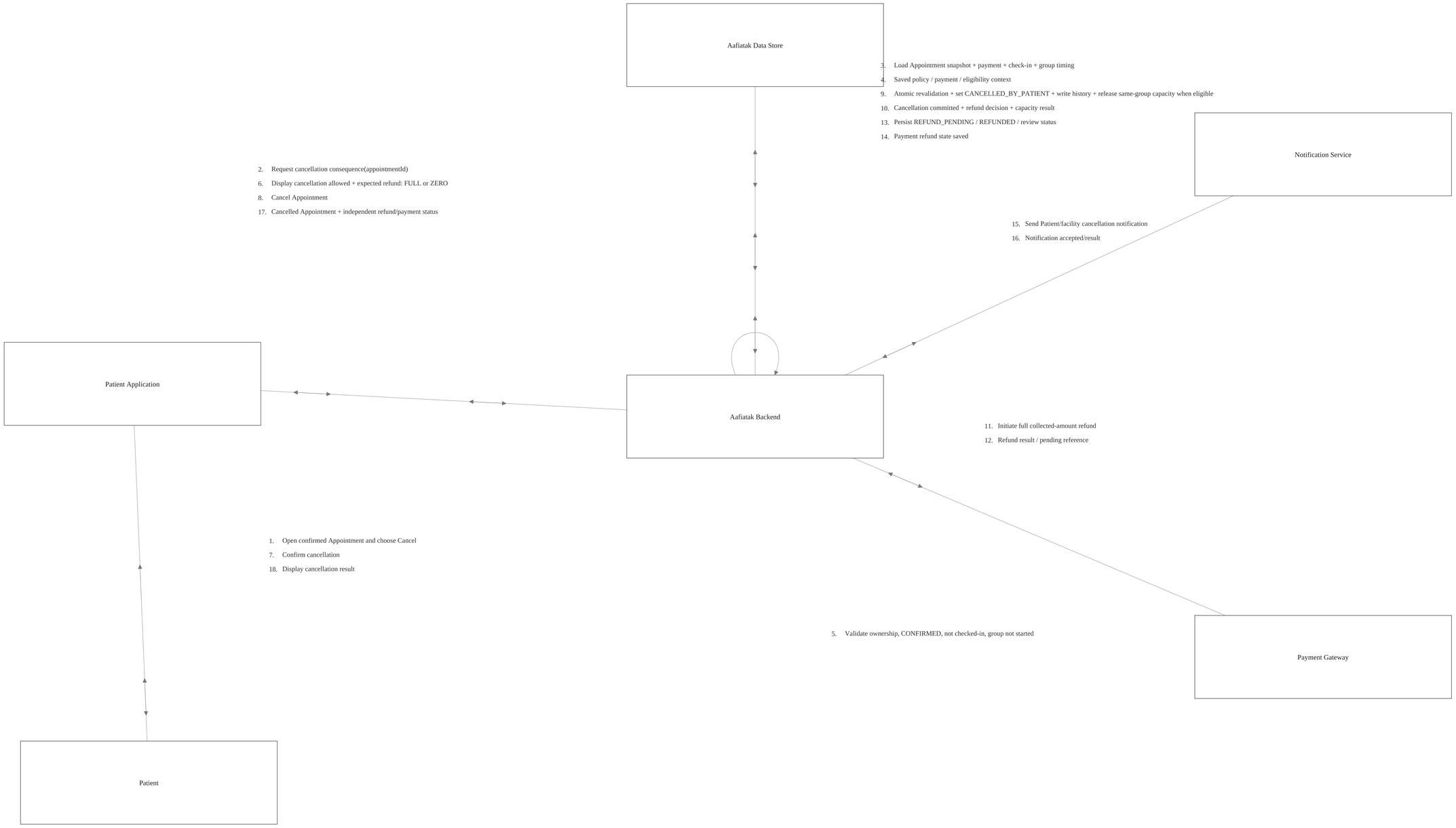
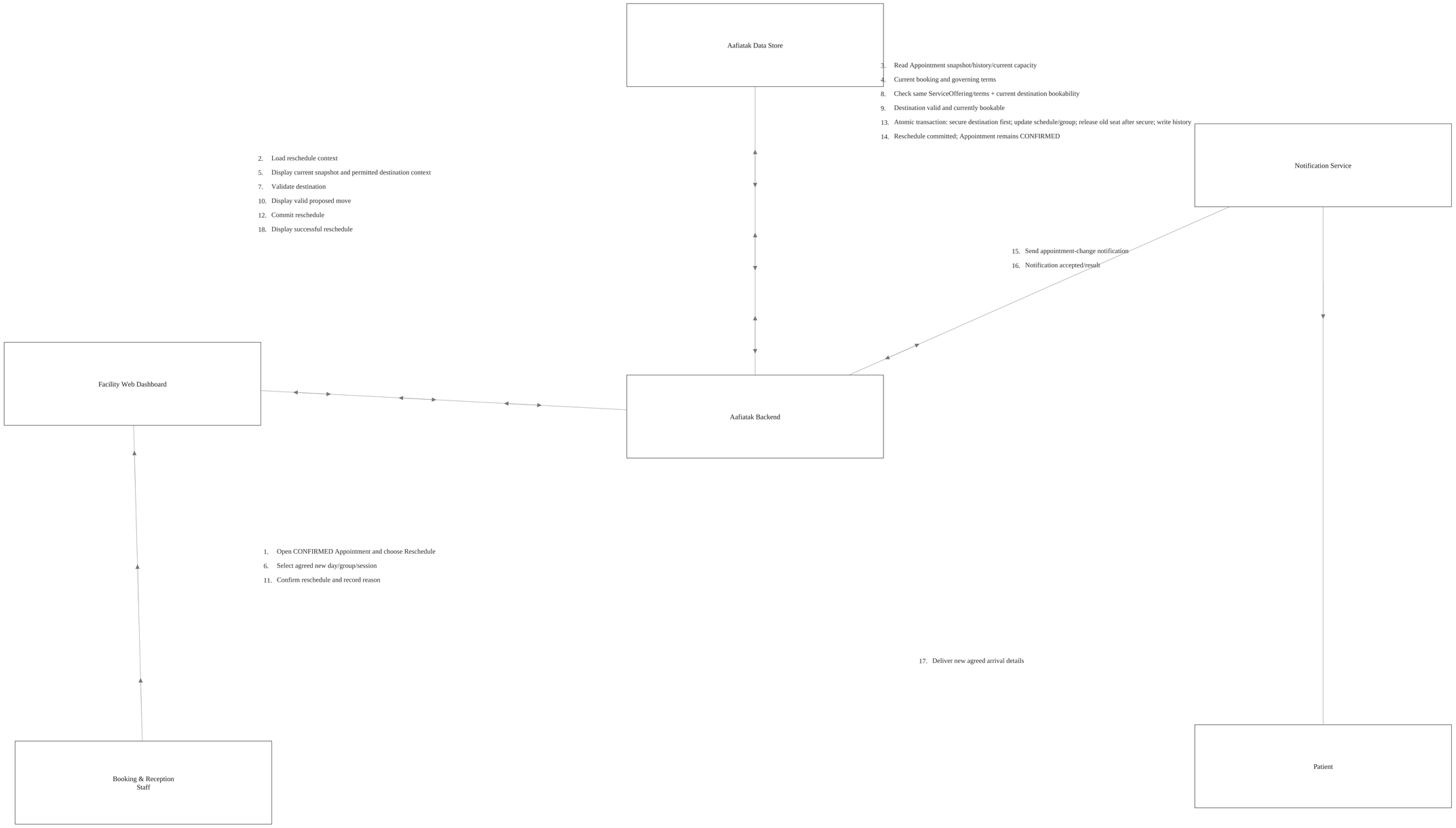
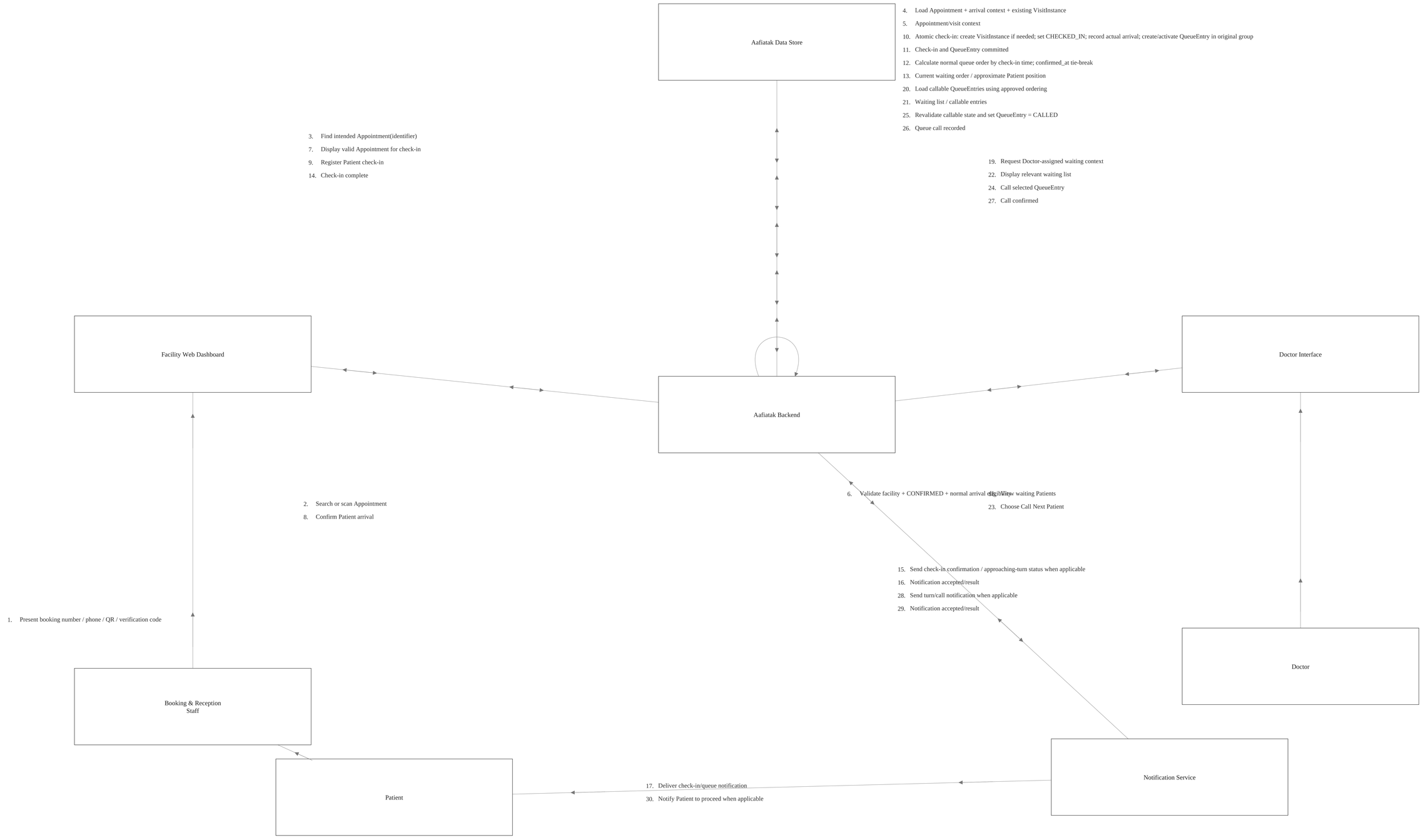
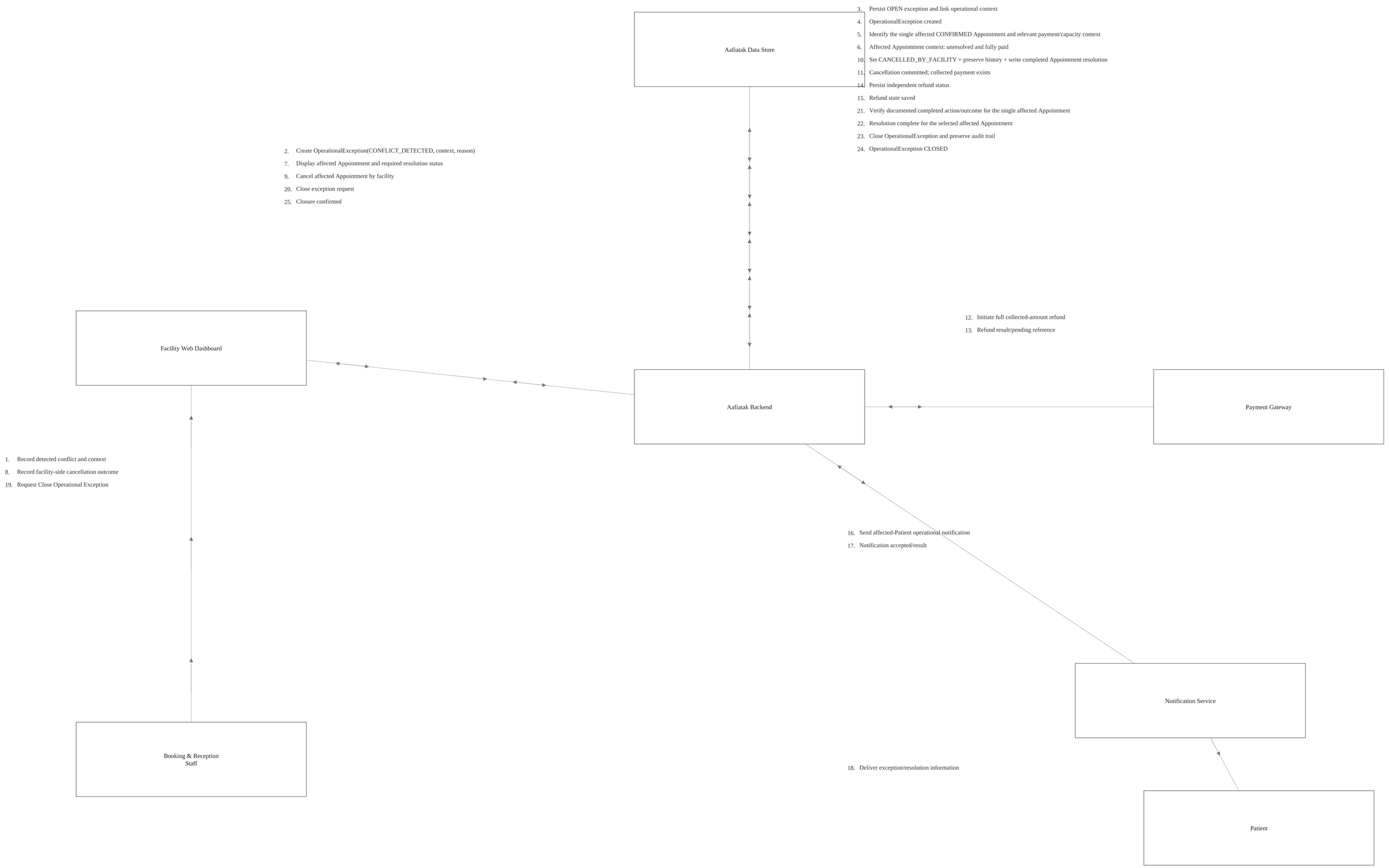

Aafiatak Data Store
4. Check phone identity uniqueness
5. Identity lookup result: available
16. Atomically create User + Patient for verified phone
17. Account/profile created exactly once
2. Start patient registration(phone)
8. Verification required
11. Verify OTP attempt
13. OTP verified; request basic Patient profile
15. Create verified Patient account/profile
18. Registration confirmed
Patient Application
Aafiatak Backend
6. Request short-lived single-use OTP delivery
7. OTP delivery request accepted
3. Normalize and validate phone number
12. Validate expiry, single-use and security limits
1. Enter phone number and choose Register
10. Submit OTP
14. Submit approved basic profile data
19. Display Patient registration success
WhatsApp Authentication Provider
Visitor
9. Deliver OTP via official WhatsApp channel


Aafiatak Data Store
3. Load PUBLISHED release + frozen booking terms
4. Release terms and current capacity state
5. Find earliest currently bookable ArrivalGroup
6. Selected group/window + remaining capacity
10. Atomically revalidate group and acquire one ACTIVE hold
11. Hold acquired: id + expiresAt + selected group
15. Load hold/snapshot and enforce one non-terminal PaymentIntent
16. Valid hold + amount/currency + PaymentIntent context
17. Create or safely reuse PaymentIntent
18. PaymentIntent ready
27. Persist SUCCEEDED and atomically revalidate hold/release/group/time
28. Payment persisted; booking target still eligible
29. Atomic commit: consume hold/capacity once + create CONFIRMED Appointment snapshot
30. Appointment confirmed; hold CONSUMED
Notification Service
31. Send full-payment booking confirmation
32. Notification accepted/result
2. Request currently bookable Aafiatak capacity
7. Display arrival window and governing booking terms
9. Create temporary ReservationHold
12. Show protected group and hold countdown
14. Start full payment for active hold
21. Continue approved gateway interaction
24. Report client return from gateway
33. Confirmed Appointment + independent SUCCEEDED payment status
Patient Application
Aafiatak Backend
19. Initiate full amount payment
20. Gateway payment session/reference
25. Verify payment through trusted gateway channel
26. Trusted payment result: SUCCESS
1. Select service / doctor context / day
8. Confirm intent to proceed
13. Proceed to full electronic payment
34. Display confirmation / receipt summary / arrival window
23. Return to application
Payment Gateway
Patient
22. Complete gateway payment interaction


Aafiatak Data Store
3. Load Appointment snapshot + payment + check-in + group timing
4. Saved policy / payment / eligibility context
9. Atomic revalidation + set CANCELLED_BY_PATIENT + write history + release same-group capacity when eligible
10. Cancellation committed + refund decision + capacity result
13. Persist REFUND_PENDING / REFUNDED / review status
14. Payment refund state saved
Notification Service
2. Request cancellation consequence(appointmentId)
6. Display cancellation allowed + expected refund: FULL or ZERO
8. Cancel Appointment
17. Cancelled Appointment + independent refund/payment status
15. Send Patient/facility cancellation notification
16. Notification accepted/result
Patient Application
Aafiatak Backend
11. Initiate full collected-amount refund
12. Refund result / pending reference
1. Open confirmed Appointment and choose Cancel
7. Confirm cancellation
18. Display cancellation result
5. Validate ownership, CONFIRMED, not checked-in, group not started
Payment Gateway
Patient


Aafiatak Data Store
3. Read Appointment snapshot/history/current capacity
4. Current booking and governing terms
8. Check same ServiceOffering/terms + current destination bookability
9. Destination valid and currently bookable
13. Atomic transaction: secure destination first; update schedule/group; release old seat after secure; write history
14. Reschedule committed; Appointment remains CONFIRMED
Notification Service
2. Load reschedule context
5. Display current snapshot and permitted destination context
7. Validate destination
10. Display valid proposed move
12. Commit reschedule
18. Display successful reschedule
15. Send appointment-change notification
16. Notification accepted/result
Facility Web Dashboard
Aafiatak Backend
1. Open CONFIRMED Appointment and choose Reschedule
6. Select agreed new day/group/session
11. Confirm reschedule and record reason
17. Deliver new agreed arrival details
Booking & Reception Staff
Patient


Aafiatak Data Store
4. Load Appointment + arrival context + existing VisitInstance
5. Appointment/visit context
10. Atomic check-in: create VisitInstance if needed; set CHECKED_IN; record actual arrival; create/activate QueueEntry in original group
11. Check-in and QueueEntry committed
12. Calculate normal queue order by check-in time; confirmed_at tie-break
13. Current waiting order / approximate Patient position
20. Load callable QueueEntries using approved ordering
21. Waiting list / callable entries
25. Revalidate callable state and set QueueEntry = CALLED
26. Queue call recorded
3. Find intended Appointment(identifier)
7. Display valid Appointment for check-in
9. Register Patient check-in
14. Check-in complete
19. Request Doctor-assigned waiting context
22. Display relevant waiting list
24. Call selected QueueEntry
27. Call confirmed
Facility Web Dashboard
Doctor Interface
Aafiatak Backend
2. Search or scan Appointment
8. Confirm Patient arrival
6. Validate facility + CONFIRMED + normal arrival eligibility
18. View waiting Patients
23. Choose Call Next Patient
15. Send check-in confirmation / approaching-turn status when applicable
16. Notification accepted/result
28. Send turn/call notification when applicable
29. Notification accepted/result
1. Present booking number / phone / QR / verification code
Doctor
Booking & Reception Staff
Notification Service
Patient
17. Deliver check-in/queue notification
30. Notify Patient to proceed when applicable


Aafiatak Data Store
3. Persist OPEN exception and link operational context
4. OperationalException created
5. Identify the single affected CONFIRMED Appointment and relevant payment/capacity context
6. Affected Appointment context: unresolved and fully paid
10. Set CANCELLED_BY_FACILITY + preserve history + write completed Appointment resolution
11. Cancellation committed; collected payment exists
14. Persist independent refund status
15. Refund state saved
21. Verify documented completed action/outcome for the single affected Appointment
22. Resolution complete for the selected affected Appointment
23. Close OperationalException and preserve audit trail
24. OperationalException CLOSED
2. Create OperationalException(CONFLICT_DETECTED, context, reason)
7. Display affected Appointment and required resolution status
9. Cancel affected Appointment by facility
20. Close exception request
25. Closure confirmed
12. Initiate full collected-amount refund
13. Refund result/pending reference
Facility Web Dashboard
Aafiatak Backend
Payment Gateway
1. Record detected conflict and context
8. Record facility-side cancellation outcome
19. Request Close Operational Exception
16. Send affected-Patient operational notification
17. Notification accepted/result
Notification Service
Booking & Reception Staff
18. Deliver exception/resolution information
Patient